Cricket Knockout Tournament Fixture Generator

A console-based program written in C that generates round-wise fixtures for a cricket knockout tournament. Teams are registered, paired using a binary tree, and the full fixture list is produced from the first round to the final.

What the program does

1. Accepts the number of teams (a power of two) and the details of each team: name, captain and city.
2. Optionally shuffles the teams for a random draw.
3. Builds a balanced binary tree in which:
   - each **leaf node** is a participating team
   - each **internal node** is a match between the winners of its two children
4. Numbers the matches round by round and prints the fixtures, for example `M5: Winner of M1 vs Winner of M2`.
5. Draws the tournament bracket as a sideways tree.

For N teams the tree always contains N leaves and **N - 1 matches**, spread over log2(N) rounds. The program does not record scores or results; it only generates the fixtures.

Files

File	Purpose
`cricket_knockout.c`	Complete source code
`sample_output.txt`	Captured run with 8 teams
`README.md`	This file

Build and run

```
gcc -Wall -std=c99 -o cricket_knockout cricket_knockout.c
./cricket_knockout
```

On Windows, run `cricket_knockout.exe`. Any standard C99 compiler works.

Menu

Option	Action
1	Register teams (count, then name, captain, city for each)
2	View the registered teams in a table
3	Generate fixtures (asks whether to use a random draw)
4	View round-wise fixtures
5	View the bracket as a tree
6	Reset and start a new tournament
0	Exit

Registration must come before fixture generation, and fixtures must be generated before they can be viewed.

Input rules

- The number of teams must be a power of two between 2 and 64.
- Team names must be unique (case-insensitive). Leaving a name blank uses a default such as `Team 3`.
- Blank captain or city fields are stored as `-`.
- Invalid numeric input is rejected and asked again.
- Once fixtures are generated, teams cannot be re-registered until you reset.

How it works

Data structures

- `Team` holds the name, captain and city of a team.
- `Node` is a tree node with `left` and `right` children, a pointer to a `Team` (set only for leaves), a `match_no` and a `height`.
- Arrays store the data: `teams[]` holds the registered teams and `matches[]` holds the match nodes in round order.

Algorithm

1. **Build the tree** - `build_tree(lo, hi)` splits the team array in half recursively. A range of one team becomes a leaf, otherwise a new internal node

is created over the two halves. A node's height is one more than its taller child, so leaves have height 0, first-round matches height 1, and the final has the greatest height.

2. **Collect the matches** - `inorder()` walks the tree and gathers every internal node into a temporary array.
3. **Order by round** - matches are copied into `matches[]` by increasing height, so round 1 comes first and the final last. Each match gets its number (`M1`, `M2`, ...) at this step.
4. **Print the fixtures** - for each round, each match shows its two sides. A side is the team name if the child is a leaf, otherwise `Winner of M<n>` referring to the child match.
5. **Draw the bracket** - `draw_bracket()` prints the right subtree, then the node, then the left subtree, indented by depth, which shows the tree rotated sideways with the final at the left edge.

Round names are chosen from the number of rounds: Final, Semi-Final and Quarter-Final for the last three rounds, and `Round n` for earlier ones.

Complexity

Building the tree, collecting the matches and printing all take O(N) time for N teams, apart from the ordering step, which is O(N log N) in the worst case because it scans the match list once per round.

Sample output

An 8-team run (no random draw):

```
Fixtures generated: 8 teams, 7 matches (N-1), 3 rounds.

--- Quarter-Final ---
  M1: India vs Australia
  M2: England vs Pakistan
  M3: South Africa vs New Zealand
  M4: Sri Lanka vs Bangladesh

--- Semi-Final ---
  M5: Winner of M1 vs Winner of M2
  M6: Winner of M3 vs Winner of M4

--- Final ---
  M7: Winner of M5 vs Winner of M6
```

The bracket view of the same tournament:

```
                  Bangladesh
            M4
                  Sri Lanka
      M6
                  New Zealand
            M3
                  South Africa
M7
                  Pakistan
            M2
                  England
      M5
                  Australia
            M1
                  India
```

The full interactive session is in `sample_output.txt`.

Limitations

- Only team counts that are powers of two are supported; there are no byes.
- At most 64 teams can be registered.
- Match results and winners are not recorded.
- Data is not saved to a file; resetting or exiting clears everything.